Guía de Estudio
Patrones de Diseño

Diseño de Software — Grado en Ingeniería Informática
Universidad de Santiago de Compostela
Curso 2025/2026

Autoría

Iago Leis Fernández
iago.leis@rai.usc.es

Nuria Guerra Casal
nuria.guerra.casal@rai.usc.es

Temas cubiertos

T8. Patrones de Creación | T9. Patrones Estructurales | T10. Patrones de Comportamiento

Aviso importante: este material es de apoyo al estudio y su contenido **no es 100% fiable**; puede contener errores, omisiones o interpretaciones imprecisas. Se prohíbe expresamente su **venta**, distribución con ánimo de lucro y su **apropiación** o presentación como obra propia.

Documento generado para preparación del ejercicio 3.

Índice

1 Cómo analizar un patrón de diseño en el examen **2**
1.1 Método de análisis en 5 pasos . . . 2
1.2 Señales visuales clave en diagramas UML . . . 3

2 Introducción a los Patrones de Diseño **4**

3 Patrones de Creación (T8) **5**
3.1 Abstract Factory (Fábrica Abstracta) . . . 5
3.1.1 Motivación y ejemplo . . . 5
3.2 Factory Method (Método de Fabricación) . . . 6
3.2.1 Motivación y ejemplo . . . 7

4 Patrones Estructurales (T9) **8**
4.1 Adapter (Adaptador) . . . 8
4.1.1 Motivación y ejemplo . . . 8
4.2 Bridge (Puente) . . . 9
4.2.1 Motivación y ejemplo . . . 9
4.3 Composite . . . 10
4.3.1 Motivación y ejemplo . . . 11
4.4 Decorator (Decorador) . . . 12
4.4.1 Motivación y ejemplo . . . 12
4.5 Proxy (Apoderado) . . . 13
4.5.1 Motivación y ejemplo . . . 14

5 Patrones de Comportamiento (T10) **15**
5.1 Observer (Observador) . . . 15
5.1.1 Motivación y ejemplo . . . 15
5.2 State (Estado) . . . 16
5.2.1 Motivación y ejemplo . . . 16
5.3 Strategy (Estrategia) . . . 17
5.3.1 Motivación y ejemplo . . . 18
5.4 Template Method (Método Plantilla) . . . 19
5.4.1 Motivación y ejemplo . . . 19

6 Tablas Resumen para el Examen **21**
6.1 Resumen general de los patrones . . . 21
6.2 Comparación de patrones similares . . . 22
6.3 Señales UML rápidas . . . 23
6.4 Tabla de participantes rápida . . . 23

Cómo analizar un patrón de diseño en el examen

Esta sección explica el **método general** para responder preguntas del tipo "identifica el patrón y explícalo". Aplícalo sistemáticamente a cada patrón.

1.1. Método de análisis en 5 pasos

Los 5 pasos para analizar cualquier patrón

1. **Identifica el patrón** — Busca en el diagrama UML las señales características: ¿hay una interfaz con subclases concretas? ¿Hay delegación? ¿Hay árbol recursivo? ¿Hay una referencia a la propia interfaz?
2. **Nombra los participantes** — Para cada clase del diagrama, asígnale el rol que juega en el patrón (p.ej., "`ConexionTCP` es el Contexto, `IEstadoTCP` es el Estado").
3. **Explica la motivación/problema** — ¿Qué problema resuelve? ¿Qué pasaría sin este patrón? (p.ej., sin State habría enormes `switch` por todo el código).
4. **Explica cuándo y cómo se usa** — Describe las condiciones de aplicación y cómo fluye la colaboración entre participantes.
5. **Indica ventajas e inconvenientes** — Menciona al menos 2 ventajas y 1 inconveniente. Si te preguntan comparaciones, aplica la tabla de la sección final.

1.2. Señales visuales clave en diagramas UML

Lo que ves en el UML	**Patrón probable**
Interfaz fábrica con múltiples fábricas concretas y múltiples familias de productos	Abstract Factory
Clase abstracta con método abstracto que devuelve un objeto de la misma jerarquía	Factory Method
Clase que implementa interfaz A y *contiene* un objeto que no implementa A	Adapter
Dos jerarquías paralelas conectadas por una referencia `-imp`	Bridge
Clase que implementa interfaz y *tiene una colección de la misma interfaz*	Composite
Clase abstracta con referencia `-componente` a la misma interfaz que implementa	Decorator
Clase que implementa interfaz y tiene referencia al "sujeto real" que también la implementa	Proxy
Clase con lista `-observadores` y método `notificar()`; observadores con `actualizar()`	Observer
Clase con referencia `-estado` a interfaz abstracta Estado; múltiples EstadosConcretos	State
Clase con referencia `-estrategia` a interfaz abstracta; múltiples EstrategiasConcreta	Strategy
Clase abstracta con método `métodoPlantilla()` no abstracto + operaciones primitivas abstractas	Template Method

Introducción a los Patrones de Diseño

Los **patrones de diseño** son soluciones reutilizables a problemas recurrentes en el diseño de software orientado a objetos. Fueron popularizados por el libro de Gamma et al. (el "Gang of Four" o GoF). Proporcionan un **vocabulario común** que eleva la abstracción y reduce la complejidad del sistema.

Categoría	Descripción
Creación	Abstraen el proceso de creación de objetos. Ocultan cómo se crean las instancias. Permiten programar para interfaces, no implementaciones.
Estructurales	Tratan la composición de clases y objetos para formar estructuras más grandes.
Comportamiento	Definen cómo interactúan y se comunican los objetos.

⋆ Recuerda para el examen

En el examen suele pedirse: **(1) identificar el patrón** en un diagrama o descripción, **(2) nombrar sus participantes**, **(3) explicar cuándo usarlo**, y **(4) compararlo con otros patrones similares**. Presta especial atención a los diagramas de estructura UML.

Patrones de Creación (T8)

Los patrones de creación permiten abstraerse de la creación de objetos, acotando el conocimiento sobre las clases concretas utilizadas. Ocultan cómo se crean y enlazan las instancias, de manera que el resto del sistema sólo conoce los objetos a través de sus interfaces.

3.1. Abstract Factory (Fábrica Abstracta)

Abstract Factory — Idea clave

Proporciona una **interfaz para crear familias de objetos relacionados** sin especificar sus clases concretas. El cliente solo conoce las interfaces de los productos, no sus implementaciones.

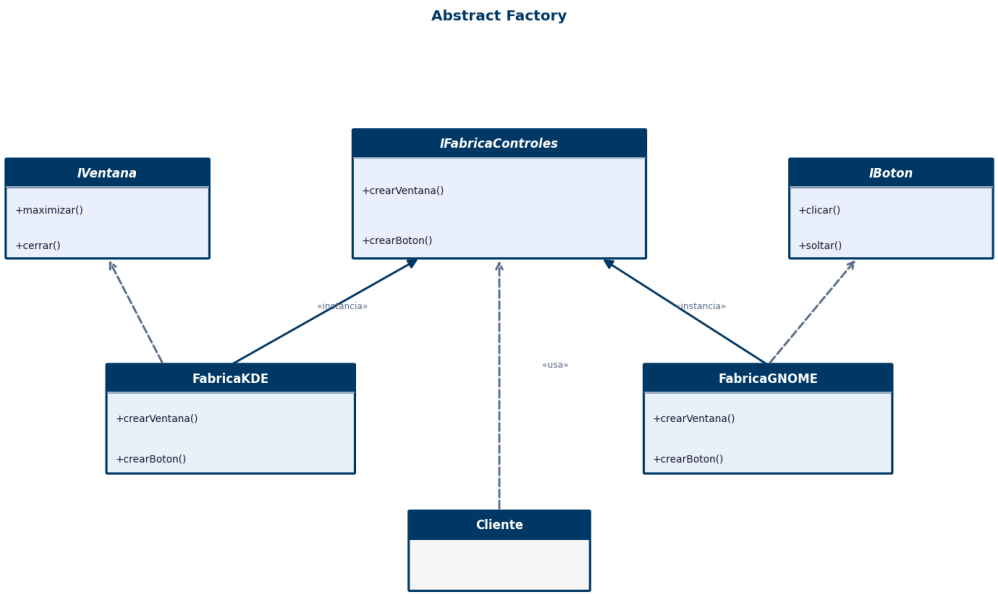

Figura 1. Diagrama UML del patrón Abstract Factory. `IFabricaControles` es la fábrica abstracta; `FabricaKDE` y `FabricaGNOME` son las fábricas concretas que crean familias de productos (`IVentana`, `IBoton`).

Motivación y ejemplo

Un toolkit gráfico que debe funcionar en KDE y en GNOME necesita crear ventanas y botones coherentes con cada entorno. Sin Abstract Factory, el código del cliente estaría lleno de condicionales `if (sistema==KDE) {...} else {...}`. Con Abstract Factory, el cliente solo habla con `IFabricaControles`: llama a `crearVentana()` y `crearBoton()` sin importarle si obtiene objetos KDE o GNOME. Cambiar de entorno es tan simple como cambiar la fábrica concreta que se le pasa al cliente.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Busca: (1) una interfaz de fábrica con *varios* métodos de creación, (2) *varias* fábricas concretas que la implementan, (3) *varias* jerarquías de productos (productos A, productos B...), (4) el cliente solo habla con la fábrica abstracta.

¿Cómo describirlo? "El patrón Abstract Factory define la interfaz `X` con métodos de creación `crearA()` y `crearB()`. Las clases `FábricaConcretas1/2` implementan dicha interfaz creando las familias `A1,B1` y `A2,B2` respectivamente. El `Cliente` usa solo la interfaz

abstracta, sin depender de clases concretas."

¿Cuándo usarlo?

- Para configurar un sistema escogiendo **una sola familia de productos** entre varias.
- Cuando cada familia está diseñada para el uso conjunto de sus productos.
- Para proporcionar bibliotecas de productos de los que solo se revelan sus interfaces.

Participantes

- **FabricaAbstracta**: declara la interfaz para crear productos abstractos.
- **FabricaConcreta**: implementa los métodos de creación para una familia específica.
- **ProductoAbstracto**: declara la interfaz de un tipo de producto.
- **ProductoConcreto**: implementa la interfaz del producto para una familia concreta.
- **Cliente**: usa solo las interfaces de fábrica y producto.

Ventajas e inconvenientes

Ventajas: desacopla clientes de implementaciones; cambio fácil de familia; garantiza consistencia entre productos de la misma familia.
Inconvenientes: difícil añadir nuevos *tipos* de productos (hay que modificar la interfaz de la fábrica y todas sus implementaciones).

3.2. Factory Method (Método de Fabricación)

Factory Method — Idea clave

Define una **interfaz para crear un objeto**, pero deja que las **subclases decidan qué clase instanciar**. Permite que una clase difiera la instanciación a sus subclases.

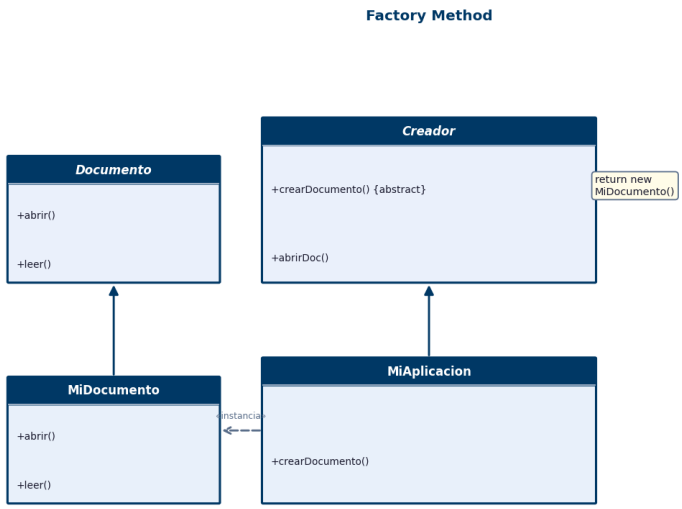

Figura 2. Diagrama UML del patrón Factory Method. `Creador` declara `crearDocumento()` abstracto; `MiAplicacion` lo implementa devolviendo `MiDocumento`. El método `abrirDoc()` usa el Factory Method sin saber la clase concreta.

Motivación y ejemplo

Una aplicación de edición de documentos tiene la clase abstracta `Aplicacion` que implementa el flujo general (`abrirDoc()`) pero no sabe qué tipo de documento crear. El método de fabricación `crearDoc()` es abstracto. La subclase `MiAplicacion` lo redefine para devolver `new MiDocumento()`. Así, el framework controla el flujo y las subclases aportan los objetos concretos. Si mañana se necesita otro tipo de documento, solo hay que crear una nueva subclase de `Aplicacion`.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Busca: (1) una clase abstracta `Creador` con un método *abstracto* que devuelve un objeto, (2) subclases concretas que redefinen ese método, (3) el método plantilla en `Creador` que usa el resultado del Factory Method.

Diferencia clave con Abstract Factory: Factory Method crea *un solo producto* mediante herencia. Abstract Factory crea *familias de productos* mediante composición.

¿Cuándo usarlo?

- Cuando una clase no puede prever la clase de objetos que debe crear.
- Cuando se quiere que las subclases especifiquen los objetos que crean.
- Para proporcionar enganches a subclases con versiones extendidas de un objeto.

Participantes

- **Producto**: define la interfaz de los objetos creados.
- **ProductoConcreto**: implementa la interfaz Producto.
- **Creador**: declara el método de fabricación; puede implementar una versión por defecto.
- **CreadorConcreto**: redefine el método de fabricación para devolver un ProductoConcreto.

Ventajas e inconvenientes

Ventajas: los clientes solo tratan con la interfaz `Producto`; permite jerarquías paralelas; dota a subclases de enganches.
Inconvenientes: los clientes pueden necesitar heredar de `Creador` solo para crear un ProductoConcreto.

⋆ Recuerda para el examen

Abstract Factory vs. Factory Method: Factory Method usa *herencia*, crea un producto, la variación está en la subclase. Abstract Factory usa *composición*, crea familias de productos, la variación está en el objeto fábrica. Abstract Factory suele implementarse internamente con varios Factory Methods.

Patrones Estructurales (T9)

Los patrones estructurales tratan la composición de clases y objetos para construir estructuras más grandes y flexibles.

4.1. Adapter (Adaptador)

Adapter — Idea clave

Convierte la **interfaz de una clase en otra** que esperan sus clientes. Permite cooperar a clases con interfaces incompatibles.

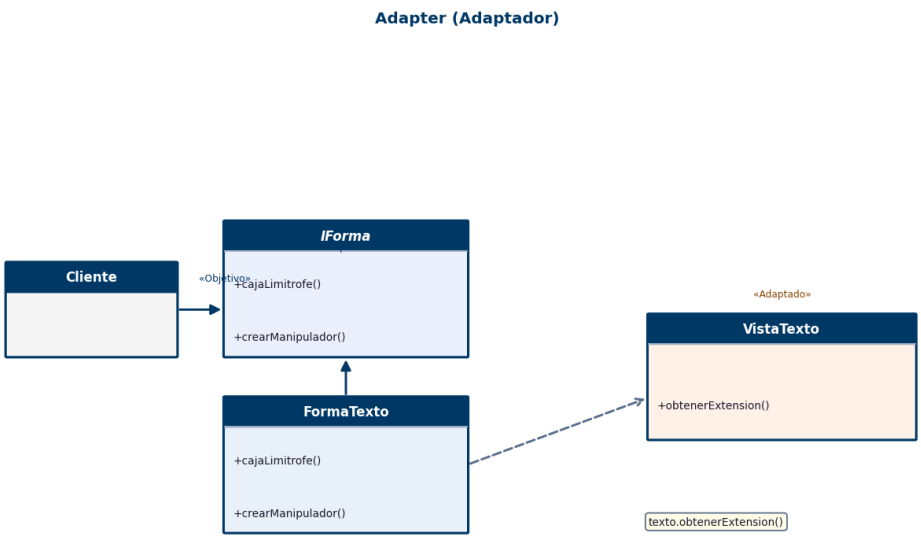

Figura 3. Diagrama UML del patrón Adapter. `FormaTexto` (Adaptador) implementa `IForma` (Objetivo) y delega internamente a `VistaTexto` (Adaptado) cuya interfaz es incompatible.

Motivación y ejemplo

El `EditorDibujo` trabaja con objetos `IForma` que tienen `cajaLimitrofe()` y `crearManipulador()`. Existe una clase de terceros `VistaTexto` que ofrece `obtenerExtension()` pero no implementa `IForma`. No podemos modificar `VistaTexto` (es de terceros). La solución es crear `FormaTexto`: implementa `IForma` y dentro llama a `texto.obtenerExtension()` para implementar `cajaLimitrofe()`. El cliente ve una `IForma` normal; no sabe nada de `VistaTexto`.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Busca: (1) una clase que implementa una interfaz (`Objetivo`), (2) esa misma clase tiene una referencia a otra clase con interfaz *distinta* (`Adaptado`), (3) los métodos del Objetivo delegan al Adaptado con traducción de llamadas.

Señal clave: el Adaptador "traduce" llamadas de una interfaz a otra. Si solo hubiera delegación a la misma interfaz, sería Proxy o Decorator.

¿Cuándo usarlo?

- Cuando se quiere usar una clase existente cuya interfaz no concuerda con la necesaria.
- Para crear una clase reutilizable que coopere con clases no previstas.
- Cuando es necesario usar varias subclases existentes sin heredar de cada una.

Participantes

- **Objetivo**: define la interfaz específica del dominio que el Cliente espera.
- **Cliente**: colabora con objetos que se ajustan a la interfaz Objetivo.
- **Adaptado**: la clase existente con interfaz incompatible que se desea reutilizar.
- **Adaptador**: implementa la interfaz Objetivo y delega al Adaptado.

Ventajas e inconvenientes

Ventajas: un mismo Adaptador puede funcionar con muchos Adaptados; permite añadir funcionalidad al adaptar.
Inconvenientes: más difícil redefinir el comportamiento del Adaptado.

4.2. Bridge (Puente)

Bridge — Idea clave

Separa una abstracción de su implementación de forma que ambas puedan variar de manera independiente. Evita la explosión de subclases.

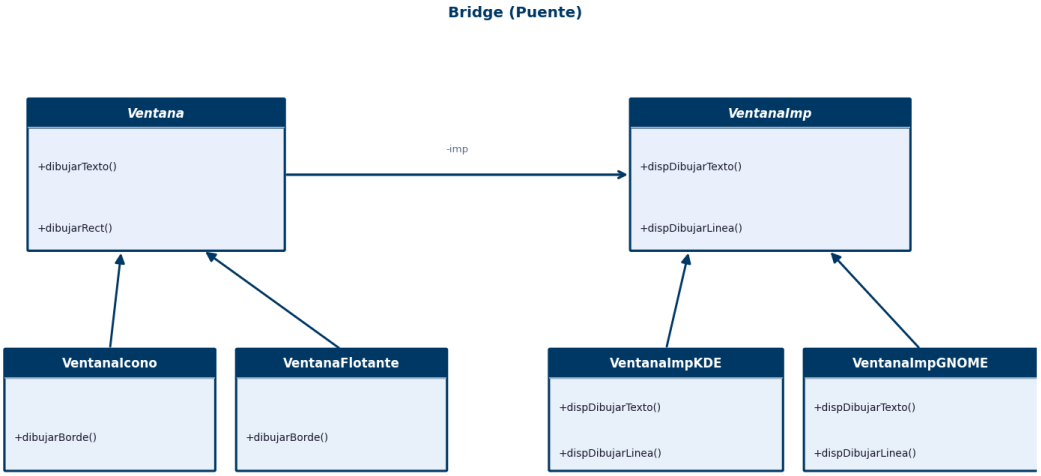

Figura 4. Diagrama UML del patrón Bridge. La jerarquía izquierda (abstracciones: `Ventana`, `VentanaIcono`, `VentanaFlotante`) y la derecha (implementaciones: `VentanaImpKDE`, `VentanaImpGNOME`) están conectadas por la referencia `-imp`.

Motivación y ejemplo

Sin Bridge, para tener ventanas de icono y flotantes en KDE y GNOME necesitaríamos 4 clases: `VentanaIconoKDE`, `VentanaIconoGNOME`, `VentanaFlotanteKDE`, `VentanaFlotanteGNOME`. Con Bridge separamos las dos dimensiones de variación: los *tipos* de ventana (abstracción) y los *sistemas gráficos* (implementación). Cada clase `Ventana` tiene una referencia `-imp` a `VentanaImp`. `dibujarRect()` llama a `imp.dispDibujarLinea()` cuatro veces. Añadir un nuevo sistema gráfico (p.ej., X11) solo requiere una nueva clase `VentanaImpX11`, sin tocar las abstracciones.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Busca: (1) dos jerarquías de herencia paralelas, (2) la jerarquía de abstracción *contiene* una referencia (`-imp`) a la interfaz raíz de la otra jerarquía, (3) los métodos de la abstracción delegan en `imp.métodoBajoNivel()`.

Diferencia clave con Adapter: Bridge se diseña *desde el principio* para separar dos dimensiones. Adapter se añade *después* para resolver una incompatibilidad existente.

¿Cuándo usarlo?

- Para evitar un vínculo permanente entre abstracción e implementación.
- Cuando abstracción e implementación deban ser ampliables independientemente.
- Para impedir que los cambios en la implementación afecten a los clientes.

Participantes

- **Abstraccion**: define la interfaz de la abstracción; mantiene referencia al Implementador.
- **AbstraccionRefinada**: extiende la interfaz de Abstraccion.
- **Implementador**: define la interfaz para las implementaciones (operaciones de bajo nivel).
- **ImplementadorConcreto**: implementa la interfaz Implementador.

Ventajas e inconvenientes

Ventajas: desacopla interfaz e implementación; la implementación se puede cambiar en tiempo de ejecución; elimina dependencias de compilación; permite desarrollo independiente.
Inconvenientes: mayor complejidad inicial en el diseño.

⋆ Recuerda para el examen

Adapter vs. Bridge: Adapter resuelve incompatibilidades *existentes* (se aplica después). Bridge planifica la separación *desde el principio*. Bridge hace que las cosas funcionen antes de ser diseñadas; Adapter lo hace después.

4.3. Composite

Composite — Idea clave

Organiza objetos en **estructuras de árbol** para representar jerarquías parte-todo. Permite tratar de manera **uniforme** a objetos individuales (hojas) y a composiciones (compuestos).

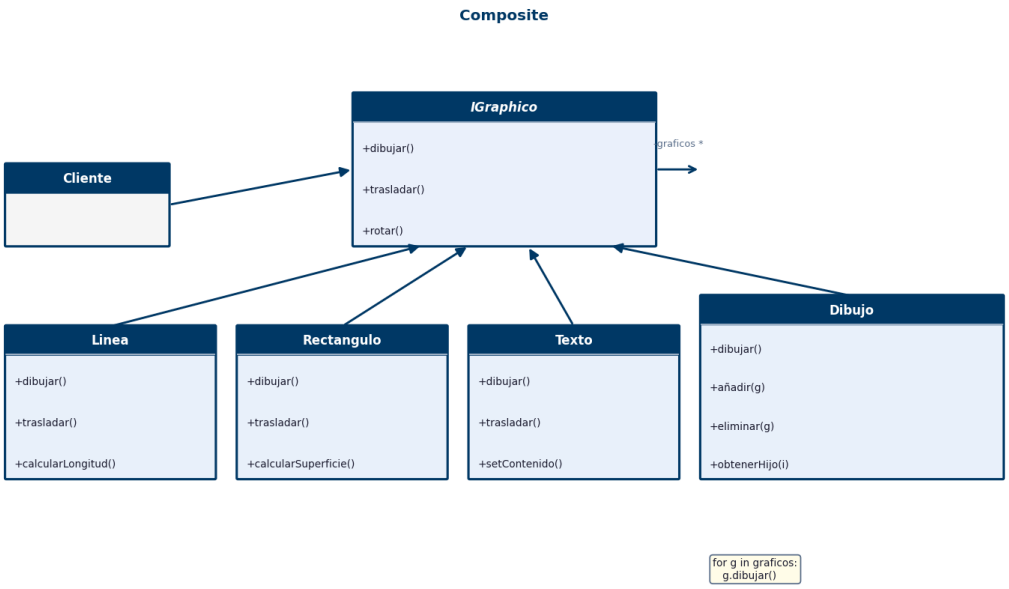

Figura 5. Diagrama UML del patrón Composite. `IGraphico` es el Componente; `Linea`, `Rectangulo` y `Texto` son Hojas; `Dibujo` es el Compuesto que agrega varios `IGraphico` y delega `dibujar()` a cada uno.

Motivación y ejemplo

En un editor gráfico, un `Dibujo` puede contener `Lineas`, `Rectangulos` y otros `Dibujos` anidados. Sin Composite, el cliente necesitaría distinguir entre objetos simples y compuestos con condicionales. Con Composite, todos implementan `IGraphico` con `dibujar()`. Cuando el cliente llama a `dibujar()` sobre un `Dibujo`, este itera sobre su colección y llama a `g.dibujar()` en cada hijo; si el hijo es otro `Dibujo`, la llamada se propaga recursivamente. El cliente solo ve `IGraphico` siempre.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? La señal más clara: **una clase (`Compuesto`) implementa la interfaz `Componente` Y tiene una colección de objetos del mismo tipo `Componente`.** Es decir, la interfaz se referencia a sí misma de forma recursiva.

¿Cómo describirlo? "`X` es el Componente (interfaz). `Y` y `Z` son Hojas que implementan `X`. `W` es el Compuesto: implementa `X` y contiene una colección de `X`; su método `operacion()` itera y delega a cada hijo."

¿Cuándo usarlo?

- Para representar jerarquías de objetos parte-todo.
- Para que los clientes puedan ignorar la diferencia entre composiciones e individuos.

Participantes

- **Componente**: declara la interfaz; implementa comportamiento predeterminado.
- **Hoja**: representa objetos primitivos sin hijos; implementa el comportamiento básico.
- **Compuesto**: almacena hijos; implementa `añadir/eliminar/obtenerHijo`; delega operaciones a hijos.
- **Cliente**: manipula objetos a través de la interfaz Componente.

Ventajas e inconvenientes

Ventajas: permite agregación recursiva; uniformiza el acceso (cliente no distingue hojas de compuestos); facilita añadir nuevos tipos de componentes.
Inconvenientes: dificulta restringir qué componentes pueden pertenecer a un grupo.

4.4. Decorator (Decorador)

Decorator — Idea clave

Asigna responsabilidades adicionales a un objeto de manera dinámica, sin modificar su clase. Alternativa flexible a la herencia para extender funcionalidad.

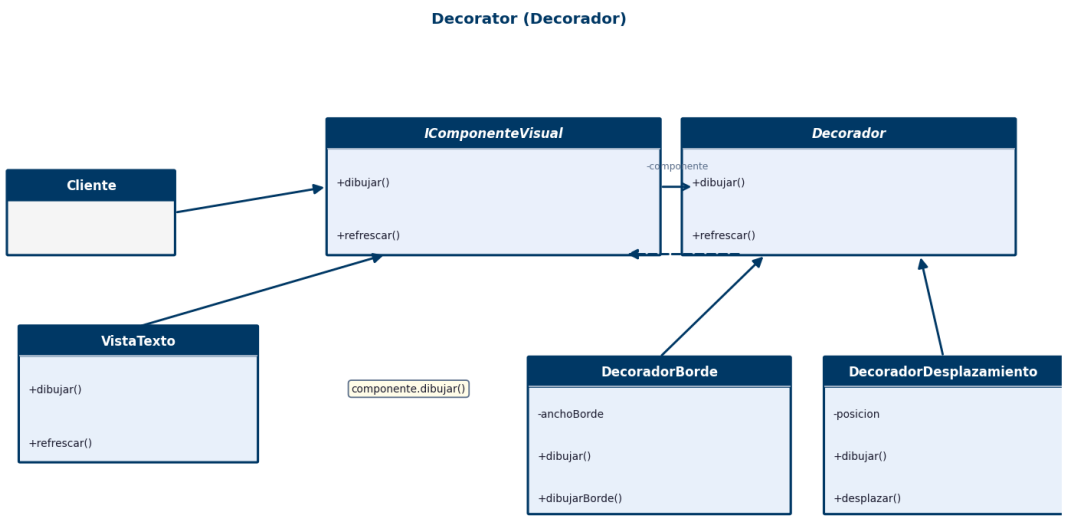

Figura 6. Diagrama UML del patrón Decorator. `Decorador` implementa `IComponenteVisual` y tiene una referencia `-componente` a la misma interfaz. Los decoradores concretos añaden su funcionalidad antes/después de delegar al componente contenido.

Motivación y ejemplo

Tenemos `VistaTexto` que muestra texto. Queremos añadirle scroll y borde de forma opcional y combinable. Si usáramos herencia, necesitaríamos `VistaTextoConScroll`, `VistaTextoConBorde` y `VistaTextoConScrollYBorde`: explosión de subclases. Con Decorator creamos `DecoradorBorde` y `DecoradorDesplazamiento` que envuelven cualquier `IComponenteVisual`. Podemos componer: `new DecoradorBorde(new DecoradorDesplazamiento(new VistaTexto()))`. Cada capa llama a `componente.dibujar()` y añade su propia lógica. Los decoradores son transparentes para el cliente.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Busca: (1) clase abstracta `Decorador` que implementa una interfaz `C` *y* tiene referencia `-componente : C`, (2) subclases concretas de `Decorador` que añaden comportamiento antes/después de delegar.

Diferencia clave con Composite: Decorator tiene exactamente un componente (no

una colección) y añade responsabilidades. Composite gestiona colecciones y representa jerarquías.
Diferencia clave con Proxy: misma estructura, pero Decorator añade responsabilidades; Proxy controla el acceso.

¿Cuándo usarlo?

- Para añadir o quitar responsabilidades a objetos individuales de forma dinámica.
- Cuando la extensión mediante herencia no es viable.

Participantes

- **Componente**: interfaz para los objetos que pueden ser decorados.
- **ComponenteConcreto**: el objeto real al que se añaden responsabilidades.
- **Decorador**: tiene referencia a un Componente y se ajusta a su interfaz.
- **DecoradorConcreto**: añade responsabilidades concretas al componente.

Ventajas e inconvenientes

Ventajas: mayor flexibilidad que la herencia; evita clases con demasiadas responsabilidades.
Inconvenientes: da lugar a muchos objetos pequeños y similares, dificulta la depuración.

4.5. Proxy (Apoderado)

Proxy — Idea clave

Proporciona un **sustituto o representante de otro objeto** para controlar el acceso a él. La interfaz del Proxy es **idéntica** a la del objeto representado.

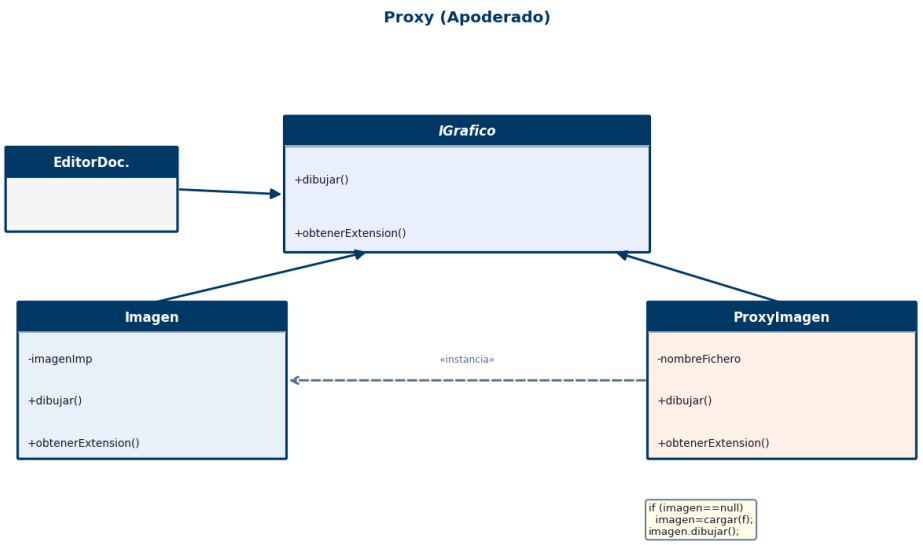

Figura 7. Diagrama UML del patrón Proxy. `ProxyImagen` implementa `IGrafico` igual que `Imagen`. El proxy crea la imagen real solo cuando es necesario (lazy loading), manteniendo la misma interfaz para el cliente.

Motivación y ejemplo

Un editor de documentos muestra muchas imágenes pero cargarlas todas al inicio sería muy lento. `ProxyImagen` implementa `IGrafico` igual que `Imagen`. Al crearse el documento, se crean proxies baratos (solo guardan el nombre de fichero). Cuando el cliente llama a `dibujar()`, el proxy verifica si la imagen real está cargada; si no, la carga en ese momento. Para `obtenerExtension()` el proxy responde directamente con la extensión guardada, sin cargar la imagen. El cliente no sabe si habla con la imagen real o con el proxy.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Busca: (1) dos clases que implementan la misma interfaz (`SujetoReal` y `Proxy`), (2) el Proxy tiene una referencia al SujetoReal y delega *selectivamente* a él (puede rechazar o retardar peticiones).

Tipos a mencionar en el examen: Proxy *remoto* (acceso a objeto en otra máquina), *virtual* (lazy loading), *de protección* (control de permisos), *referencia inteligente* (conteo de referencias, logging).

Participantes

- **Sujeto**: define la interfaz común para SujetoReal y Proxy.
- **SujetoReal**: el objeto real que el Proxy representa.
- **Proxy**: controla el acceso al SujetoReal; puede gestionar su creación/destrucción.

Ventajas e inconvenientes

Ventajas: Proxy remoto oculta la ubicación del objeto; Proxy virtual crea por encargo; Proxy de protección regula acceso; permite creación diferida.
Inconvenientes: añade una capa de indirección que puede afectar al rendimiento.

⋆ Recuerda para el examen

Adapter / Proxy / Decorator — las tres diferencias críticas:

- **Adapter**: cambia la *interfaz*. El adaptado tiene interfaz distinta al objetivo.
- **Proxy**: misma interfaz, controla el *acceso*. El proxy puede rechazar o retardar peticiones.
- **Decorator**: misma interfaz, añade *responsabilidades*. Se pueden encadenar múltiples capas.

Patrones de Comportamiento (T10)

5.1. Observer (Observador)

Observer — Idea clave

Define una **dependencia uno-a-muchos** entre objetos: cuando el sujeto cambia su estado, todos sus observadores son **notificados y actualizados automáticamente**.

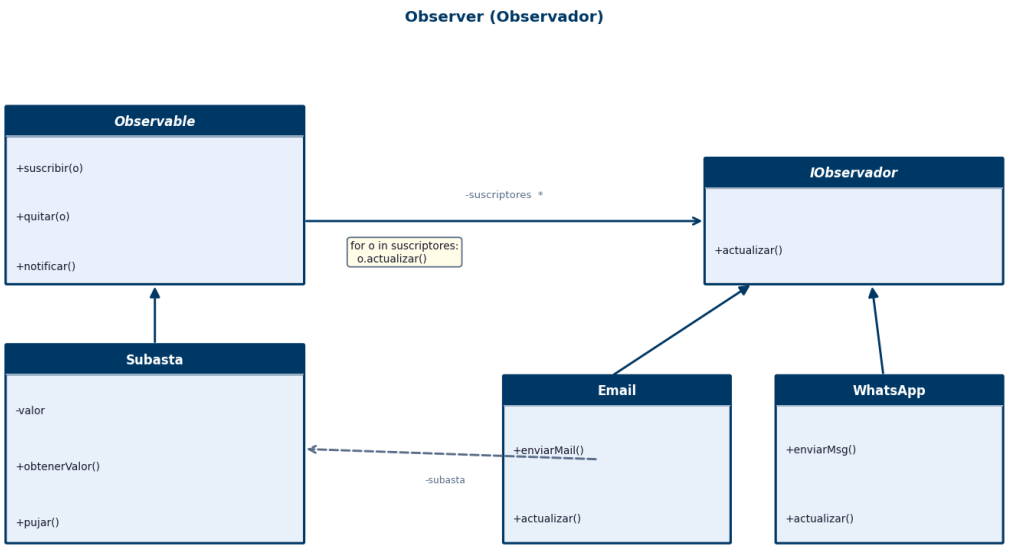

Figura 8. Diagrama UML del patrón Observer. `Subasta` (SujetoConcreto) hereda de `Observable` y notifica a `Email` y `WhatsApp` (ObservadoresConcretos) cuando cambia su valor. Estos consultan el nuevo valor llamando a `obtenerValor()`.

Motivación y ejemplo

En una subasta online, cuando alguien puja (`pujar()`), el sistema debe notificar a todos los interesados: enviar un email de confirmación y un mensaje de WhatsApp. Sin Observer, `Subasta` necesitaría conocer directamente `Email` y `WhatsApp`: acoplamiento fuerte. Con Observer, `Subasta` hereda de `Observable` y llama a `notificar()` tras cada puja. El método `notificar()` itera sobre la lista de suscriptores y llama a `o.actualizar()` en cada uno. Cada observador consulta el nuevo valor con `subasta.obtenerValor()`. Añadir un nuevo tipo de notificación (p.ej., SMS) no requiere modificar `Subasta`.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Busca: (1) clase con lista de observadores y método `notificar()`/`notifyAll()`, (2) interfaz `Observador` con método `actualizar()`, (3) los ObservadoresConcretos consultan el estado del sujeto en `actualizar()`.

Flujo a describir: "El SujetoConcreto cambia su estado → llama a `notificar()` → cada observador recibe `actualizar()` → consulta el nuevo estado con `obtenerEstado()`."

¿Cuándo usarlo?

- Cuando un cambio en un objeto requiere modificar otros sin importar cuántos son.
- Cuando un objeto debe poder notificar a otros sin conocer quiénes son.

Participantes

- **Sujeto**: conoce sus observadores; interfaz para suscribirse/desuscribirse.
- **Observador**: define la interfaz de actualización (`actualizar()`).
- **SujetoConcreto**: almacena el estado de interés; notifica los cambios.
- **ObservadorConcreto**: mantiene referencia al sujeto concreto; sincroniza su estado.

Ventajas e inconvenientes

Ventajas: acoplamiento mínimo; comunicación por difusión; modificación independiente de sujetos y observadores.
Inconvenientes: posibles actualizaciones en cascada inesperadas; el protocolo simple no detalla qué cambió.

5.2. State (Estado)

State — Idea clave

Permite que un objeto **modifique su comportamiento cuando cambia su estado interno**. Parecerá que el objeto cambia de clase.

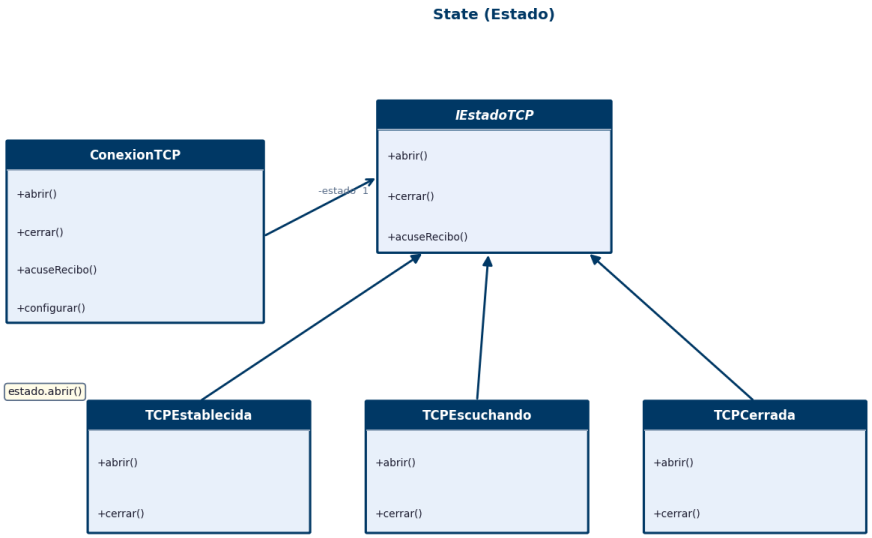

Figura 9. Diagrama UML del patrón State. `ConexionTCP` (Contexto) delega en su objeto `-estado` actual. Los estados concretos (`TCPEstablecida`, `TCPEscuchando`, `TCPCerrada`) implementan comportamientos distintos para las mismas operaciones.

Motivación y ejemplo

Una conexión TCP tiene tres estados: Establecida, Escuchando y Cerrada. Sin State, `ConexionTCP.abrir()` tendría un `switch(estadoActual)` con tres bloques grandes, repetido en `cerrar()` y `acuseDeRecibo()`. Con State, `ConexionTCP` guarda una referencia `-estado` a la interfaz `IEstadoTCP`. Cada método simplemente delega: `estado.abrir()`. El objeto estado actual (p.ej., `TCPCerrada`) implementa

el comportamiento correcto y puede cambiar el estado del contexto (`contexto.setEstado(new TCPEstablecida())`). Añadir un nuevo estado solo requiere una nueva clase.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Busca: (1) Contexto con referencia `-estado` a interfaz abstracta, (2) métodos del Contexto que delegan en `estado.metodo()`, (3) múltiples EstadosConcretos que implementan la interfaz.

Diferencia clave con Strategy: en State, **el propio estado cambia el siguiente estado** (las transiciones son internas al patrón). En Strategy, el cliente externo elige y cambia la estrategia.

¿Cuándo usarlo?

- Cuando el comportamiento depende del estado y cambia en tiempo de ejecución.
- Cuando hay muchas estructuras condicionales dependientes del estado.

Participantes

- **Contexto**: mantiene instancia de EstadoConcreto que define el estado actual.
- **Estado**: interfaz que encapsula el comportamiento de cada estado.
- **EstadoConcreto**: implementa el comportamiento de un estado particular.

Ventajas e inconvenientes

Ventajas: concentra comportamiento por estado; facilita añadir estados; elimina condicionales; hace explícitas las transiciones.
Inconvenientes: incrementa el número de clases.

5.3. Strategy (Estrategia)

Strategy — Idea clave

Define una **familia de algoritmos**, los encapsula en objetos y los hace **intercambiables**. El algoritmo varía con independencia de los clientes.

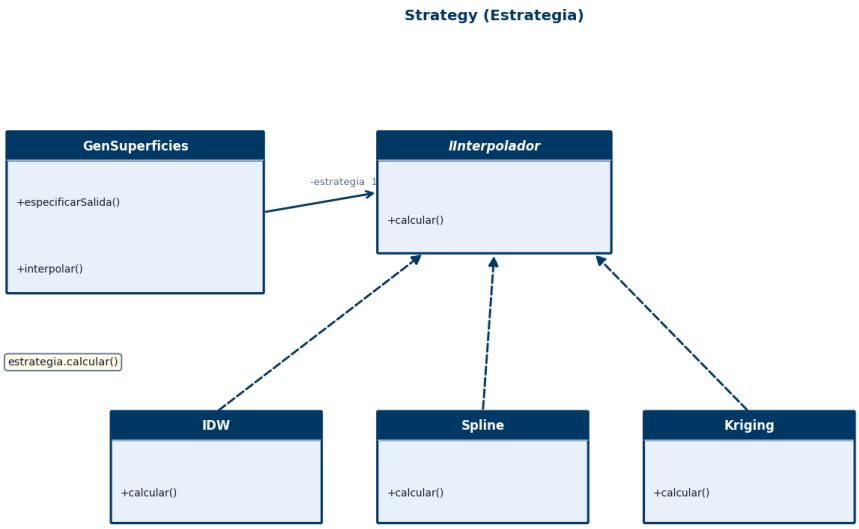

Figura 10. Diagrama UML del patrón Strategy. `GenSuperficies` (Contexto) tiene referencia `-estrategia` a `IInterpolador`. Las estrategias concretas (`IDW`, `Spline`, `Kriging`) son intercambiables sin modificar el contexto.

Motivación y ejemplo

Un generador de superficies puede interpolar datos usando tres algoritmos: IDW (rápido pero menos preciso), Spline (preciso, moderado) y Kriging (muy preciso, lento). Sin Strategy, `interpolar()` tendría un `switch` enorme. Con Strategy, el algoritmo se encapsula en objetos que implementan `IInterpolador.calcular()`. `GenSuperficies` delega: `estrategia.calcular()`. El cliente puede cambiar la estrategia en tiempo de ejecución sin tocar `GenSuperficies`. Incluso puede almacenar estrategias y reutilizarlas.

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Muy similar a State estructuralmente. La diferencia semántica: Strategy se usa cuando hay *algoritmos alternativos* elegidos externamente por el cliente. State se usa cuando el *comportamiento varía según el estado interno* del objeto.

Señal práctica: si las "estrategias" se conocen entre sí o pueden cambiar de una a otra solas → es State. Si son independientes y el cliente elige → es Strategy.

¿Cuándo usarlo?

- Cuando existen varias clases relacionadas que solo difieren en su comportamiento.
- Cuando se necesitan variantes de un algoritmo.
- Para eliminar condicionales largas de selección de comportamiento.

Participantes

- **Estrategia**: declara la interfaz común a todos los algoritmos.
- **EstrategiaConcreta**: implementa un algoritmo particular.
- **Contexto**: mantiene referencia a la Estrategia; delega en ella el algoritmo.

Ventajas e inconvenientes

Ventajas: familia de algoritmos intercambiables; modifica algoritmos sin afectar al contexto; elimina condicionales.

Inconvenientes: los clientes deben conocer las estrategias; estrategias simples pueden no usar toda la información que reciben.

⋆ Recuerda para el examen

State vs. Strategy — resumen definitivo:

	State	Strategy
¿Quién cambia?	El propio estado (interno)	El cliente (externo)
¿Los objetos se conocen?	Sí (transiciones entre estados)	No (independientes)
¿Propósito?	Variar comportamiento por estado	Intercambiar algoritmo
¿Cuántos activos?	Uno a la vez	Uno a la vez

5.4. Template Method (Método Plantilla)

Template Method — Idea clave

Define el **esqueleto de un algoritmo en una clase base**, delegando algunos pasos a las subclases. Permite redefinir ciertos pasos sin cambiar la estructura global.

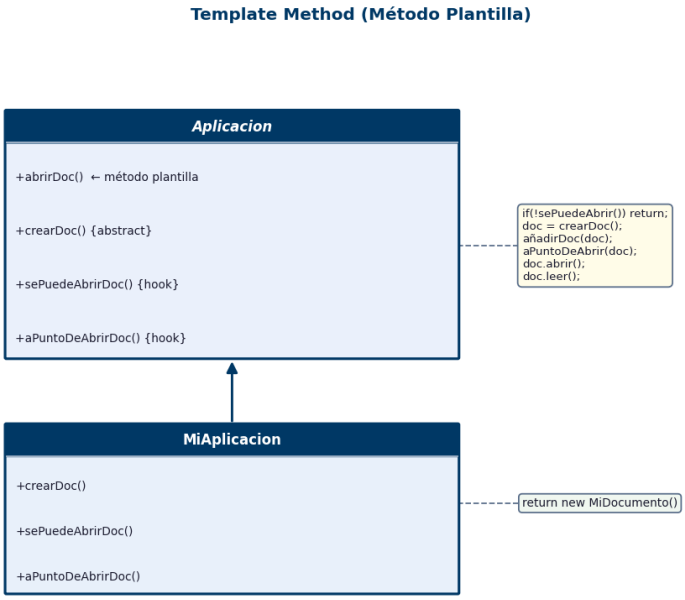

Figura 11. Diagrama UML del patrón Template Method. `Aplicacion` define el método plantilla `abrirDoc()` con el esqueleto del algoritmo. `MiAplicacion` implementa las operaciones primitivas abstractas (`crearDoc()`) y puede sobreescribir los hooks (`sePuedeAbrirDoc()`).

Motivación y ejemplo

Un framework de aplicaciones define el método `abrirDoc()` con estos pasos: (1) comprobar si se puede abrir, (2) crear el documento, (3) añadirlo a la lista, (4) notificar que va a abrirse, (5) abrirlo, (6) leerlo. Los pasos 1, 4 son **hooks** (métodos vacíos por defecto, opcionales); el paso 2 es **abstracto** (obligatorio). `MiAplicacion` redefine `crearDoc()` devolviendo `new MiDocumento()` y

opcionalmente sobreescribe los hooks. El framework llama al código de las subclases, no al revés: esto es la *Inversión de Control* ("Hollywood Principle: Don't call us, we'll call you").

Guía de análisis: ¿cómo identificar y describir este patrón?

¿Cómo identificarlo? Busca: (1) clase abstracta con un método *concreto* (no abstracto) que llama a *varios* métodos abstractos y/o de enganche, (2) subclases que implementan esos métodos primitivos.

Diferencia clave con Strategy: Template Method usa *herencia* para variar *pasos* del algoritmo; la variación se fija en compilación. Strategy usa *composición* para intercambiar el *algoritmo completo*; la variación ocurre en ejecución.

Tipos de operaciones: Primitivas abstractas (obligatorio implementar), **concretas en ClaseAbstracta** (no tocar), **hooks/enganche** (opcional sobreescribir, vacías por defecto).

¿Cuándo usarlo?

- Para implementar las partes invariantes de un algoritmo dejando las variables a las subclases.
- Para factorizar comportamiento repetido en una clase común.
- Para controlar extensiones de subclases mediante operaciones de enganche.

Participantes

- **ClaseAbstracta**: implementa el método plantilla; declara operaciones primitivas y hooks.
- **ClaseConcreta**: implementa las operaciones primitivas para completar el algoritmo.

Ventajas e inconvenientes

Ventajas: reutilización del código del algoritmo; inversión de control.
Inconvenientes: puede ser difícil de mantener con muchos pasos; las subclases solo pueden variar dentro del esqueleto definido.

⋆ Recuerda para el examen

Template Method vs. Strategy:

	Template Method	**Strategy**
Mecanismo	Herencia	Composición
Granularidad	Pasos del algoritmo	Algoritmo completo
Variación	En compilación (subclase)	En ejecución (objeto)
Variante de código	Métodos primitivos	Objeto estrategia

Tablas Resumen para el Examen

6.1. Resumen general de los patrones

Patrón	Categoría	Idea clave	Cuándo usarlo
Abstract Factory	Creación	Interfaz para crear familias de objetos. Cliente solo conoce interfaces.	Familias de productos que deben usarse juntos. Cambio de familia en ejecución.
Factory Method	Creación	Subclase decide qué clase instanciar. Método abstracto que devuelve un Producto.	Clase no puede prever qué objeto crear. Dejar a subclases la elección.
Adapter	Estructural	Convierte interfaz incompatible en la que espera el cliente.	Reutilizar una clase de terceros cuya interfaz no encaja.
Bridge	Estructural	Separa abstracción de implementación en dos jerarquías independientes.	Evitar explosión de subclases por dos dimensiones de variación.
Composite	Estructural	Árbol de objetos. Hojas y compuestos tienen la misma interfaz.	Jerarquías parte-todo; cliente no distingue hoja de compuesto.
Decorator	Estructural	Añade responsabilidades dinámicamente sin subclasificar.	Extensión flexible de objetos en tiempo de ejecución.
Proxy	Estructural	Sustituto con la misma interfaz para controlar el acceso.	Acceso remoto, lazy loading, control de permisos.
Observer	Comportamiento	Notificación automática 1:N cuando el sujeto cambia.	Mantener consistencia entre objetos dependientes sin acoplarlos.
State	Comportamiento	Delega en objeto-estado; cambia de comportamiento según estado.	Comportamiento dependiente del estado; eliminar condicionales de estado.
Strategy	Comportamiento	Familia de algoritmos intercambiables encapsulados en objetos.	Variantes de un algoritmo; eliminar condicionales de selección.
Template Method	Comportamiento	Esqueleto de algoritmo en clase base; subclases rellenan los pasos.	Partes fijas y variables de un algoritmo; factorizar código común.

6.2. Comparación de patrones similares

Comparativa completa: Adapter / Proxy / Decorator

	Adapter	**Proxy**	**Decorator**
Interfaz al cliente	Distinta (nueva)	Idéntica al original	Idéntica al original
Propósito	Compatibilizar interfaces	Controlar acceso	Añadir responsabilidades
Conoce al original	Sí (lo adapta)	Sí (lo representa)	Sí (lo decora)
Encadenamiento	No	No	Sí (capas)
Momento de diseño	Después (remedio)	Antes o después	Antes o después

Comparativa: Abstract Factory vs. Factory Method

	Abstract Factory	**Factory Method**
Mecanismo	Composición/delegación	Herencia
Qué crea	Familia de productos	Un producto
Quién decide	Objeto fábrica	Subclase
Complejidad	Mayor	Menor
Cambio en ejecución	Sí (cambiar fábrica)	No (cambiar subclase)

Comparativa: State vs. Strategy vs. Template Method

	State	**Strategy**	**Template Method**
Mecanismo	Composición	Composición	Herencia
Qué varía	Comportamiento completo	Algoritmo completo	Pasos del algoritmo
Quién cambia	El estado (interno)	El cliente (externo)	La subclase
Se conocen entre sí	Sí	No	N/A
Variación	En ejecución	En ejecución	En compilación

6.3. Señales UML rápidas

Identificación rápida por estructura UML

- **Clase implementa interfaz Y tiene colección de la misma interfaz → Composite**
- **Clase abstracta implementa interfaz Y tiene referencia 1:1 a la misma interfaz → Decorator**
- **Dos clases implementan la misma interfaz; una referencia a la otra → Proxy**
- **Clase implementa interfaz A Y contiene objeto de interfaz B (distinta) → Adapter**
- **Dos jerarquías paralelas conectadas por referencia `-imp` → Bridge**
- **Contexto con `-estado` y múltiples clases `EstadoConcreto` → State**
- **Contexto con `-estrategia` y múltiples `EstrategiaConcreta` → Strategy**
- **Clase abstracta con método concreto que llama a métodos abstractos suyos → Template Method**
- **Clase con lista `-observadores` y método `notificar()` → Observer**
- **Interfaz fábrica con varios métodos `crear...()` y varias fábricas concretas → Abstract Factory**
- **Clase abstracta con método abstracto que devuelve objeto de jerarquía propia → Factory Method**

6.4. Tabla de participantes rápida

Patrón	Participantes
Abstract Factory	**FabricaAbstracta**, FabricaConcreta, ProductoAbstracto, ProductoConcreto, Cliente
Factory Method	Producto, ProductoConcreto, **Creador**, **CreadorConcreto**
Adapter	Objetivo, Cliente, **Adaptado**, **Adaptador**
Bridge	**Abstraccion**, AbstraccionRefinada, **Implementador**, ImplementadorConcreto
Composite	**Componente**, **Hoja**, **Compuesto**, Cliente
Decorator	Componente, ComponenteConcreto, **Decorador**, **DecoradorConcreto**
Proxy	Sujeto, **SujetoReal**, **Proxy**
Observer	**Sujeto**, **Observador**, SujetoConcreto, ObservadorConcreto
State	**Contexto**, **Estado**, EstadoConcreto
Strategy	**Estrategia**, EstrategiaConcreta, Contexto
Template Method	**ClaseAbstracta**, ClaseConcreta

⋆ Recuerda para el examen

Preguntas tipo del examen:

1. Dado un diagrama UML, identifica el patrón y nombra los participantes.
2. Explica la diferencia entre los patrones X e Y (frecuente: Decorator/Proxy, State/Strategy, Adapter/Bridge, Template Method/Strategy).
3. ¿En qué situación usarías el patrón X? Razona tu respuesta.
4. Indica las ventajas e inconvenientes del patrón X.
5. Dado un problema de diseño, ¿qué patrón(es) aplicarías?

Referencia principal: Gamma, Helm, Johnson, Vlissides. *Design Patterns*. Addison-Wesley, 1995.
Diapositivas de José Varela Pet, Departamento de Electrónica y Computación, USC, 2025/2026.